SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I C. PRIYANKA / 192511074 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **C. PRIYANKA**

Annexure A
Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)
A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.

ANSWER:

1. Introduction

SHA (Secure Hash Algorithm) is a cryptographic hash function that converts input data into a fixed-length hash value. SHA-256 produces a 256-bit hash. It is used for data integrity, digital signatures, and password security systems.

2. Identify the Errors

Common errors in a SHA implementation include:

- Incorrect message padding.
- Incorrect calculation of the message length.
- Improper processing of 512-bit blocks.
- Incorrect bitwise operations and rotations.
- Wrong byte order (endianness).

These errors can produce incorrect hash values.

3. Fix the Errors

Message padding: SHA-256 appends a 1 bit, followed by enough 0 bits and a 64-bit representation of the original message length.

Block processing: The padded message is divided into 512-bit blocks, and each block is processed using the SHA-256 compression function.

Bitwise operations: Use correct XOR, AND, NOT, and right-rotation operations with 32-bit masking.

Endianness: SHA-256 interprets message words in big-endian order.

4. Optimize Processing Speed

For files larger than 50 MB:

- Read data in chunks instead of loading the entire file into memory.
- Use Python's built-in hashlib module.
- Avoid unnecessary data copying.
- Use efficient file-reading operations.

Example Python code:

```
import hashlib



def hash_file(filename):

    sha = hashlib.sha256()

    with open(filename, "rb") as f:

        for chunk in iter(lambda: f.read(1024 * 1024), b""):

            sha.update(chunk)

    return sha.hexdigest()



print(hash_file("largefile.bin"))
```

5. Security Analysis

A checksum such as CRC32 detects accidental data corruption but is not designed to resist malicious modification. SHA-256 provides stronger integrity verification because finding collisions or constructing matching hashes is computationally difficult.

However, a plain hash does not authenticate the sender. For authentication, use a keyed hash such as HMAC or a digital signature.

Conclusion

Correct padding, block processing, bitwise operations, and endianness are essential for accurate SHA implementation. Using hashlib improves performance and reduces implementation errors.

---

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation
A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.

ANSWER:

1. Introduction

SSL/TLS protocols protect communication between clients and servers. TLS provides confidentiality, integrity, and authentication. Modern systems use TLS rather than the outdated SSL protocols.

2. Identify the Errors

Common causes of handshake failures include:

- Expired or invalid certificates.
- Incorrect hostname verification.
- Untrusted certificate authorities.
- Incompatible TLS versions or cipher suites.
- Incorrect key exchange parameters.

Handshake delays may result from repeated connections and unnecessary certificate verification or network round trips.

3. Fix the Errors

- Verify certificate validity and hostname.
- Use trusted certificate authorities.
- Enable modern TLS versions and secure cipher suites.
- Use authenticated key exchange.
- Handle connection errors and timeouts properly.

4. Optimize Handshake Performance

- Use TLS session resumption when supported.
- Reuse established secure connections.
- Reduce unnecessary network round trips.
- Avoid repeated connection establishment.
- Use efficient cryptographic libraries.

Example Python code:

```
import ssl

import socket
```

```
context = ssl.create_default_context()


with socket.create_connection(("example.com", 443)) as sock:
    with context.wrap_socket(sock, server_hostname="example.com") as tls:
        print("TLS version:", tls.version())
        print("Cipher:", tls.cipher())
```

This example establishes a TLS connection using certificate verification provided by Python's default SSL context.

5. Performance vs Security

Session resumption can reduce handshake latency and computational overhead. However, implementations must manage session tickets and resumption state securely.

Disabling certificate verification may make connections faster in some circumstances, but it creates serious security risks and should never be used as an optimization.

Conclusion

TLS performance can be improved through secure session resumption, connection reuse, and efficient cryptographic libraries without sacrificing certificate validation.

---

3. Debugging and Optimizing Digital Signature Algorithm (DSA)
A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.

ANSWER:

1. Introduction

The Digital Signature Algorithm (DSA) is a public-key cryptographic algorithm used to generate and verify digital signatures. It provides authentication and integrity.

2. Identify the Errors

Common errors include:

- Invalid domain parameters.
- Incorrect private or public keys.
- Weak or repeated random values.
- Incorrect modular arithmetic.
- Incorrect signature verification.

3. Correct Signature Generation

DSA uses the following parameters:

- p: a large prime number.
- q: a prime divisor of $p - 1$.
- g: a generator.
- x: private key.
- $y = g^x p$: public key.

For a message hash $H(m)$, signature generation calculates:

$$r = (g^k p)\, q$$
$$s = k^{-1}(H(m) + xr)\, q$$

Here, $k$is a secret, securely generated per-signature nonce.

4. Signature Verification

Calculate:

$$w = s^{-1}\, q$$
$$u_1 = H(m)w\, q$$
$$u_2 = rw\, q$$

Then calculate:

$$v = \left((g^{u_1} y^{u_2} p)\, q\right)$$

The signature is valid if:

$$\boxed{v = r}$$

The signature must also satisfy $0 < r < q$and $0 < s < q$.

5. Optimize Performance and Security

- Use efficient modular exponentiation.
- Use trusted cryptographic libraries.
- Validate all public parameters.
- Generate a unique, unpredictable nonce for every signature.
- Never reuse or expose the nonce.

Poor nonce generation can reveal the private key. In addition, DSA is no longer approved for generating new signatures under current NIST digital-signature guidance, so modern implementations should generally use approved alternatives such as RSA or ECDSA where appropriate.

Conclusion

Correct parameters, modular arithmetic, and secure nonce generation are essential for signature validity and security. Trusted cryptographic libraries help prevent implementation errors.

---

4. Debugging Key Management System
A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).
Optimize secure storage using encryption and access control mechanisms.
Analyze how poor key management can compromise an entire cryptographic system.

ANSWER:

1. Introduction

A Key Management System (KMS) manages cryptographic keys throughout their lifecycle. It supports secure key generation, storage, distribution, rotation, and destruction.

2. Identify the Errors

Common key management problems include:

- Storing keys in plaintext.
- Hardcoding keys in source code.
- Using weak random number generators.
- Sharing keys through insecure channels.
- Failing to rotate or revoke compromised keys.
- Allowing unauthorized users to access keys.

3. Fix Key Storage

Keys should be encrypted at rest and protected by access controls. Use a dedicated key management service, hardware security module (HSM), or operating-system credential store where appropriate.

A common approach is **envelope encryption**:

1. Generate a data encryption key (DEK).
2. Encrypt the data using the DEK.
3. Encrypt the DEK using a key-encryption key (KEK).
4. Store the encrypted data and encrypted DEK.
5. Decrypt the DEK only when an authorized application needs it.

4. Improve Key Lifecycle Management

- **Generation:** Use a cryptographically secure random number generator.
- **Distribution:** Transmit keys through secure, authenticated channels.
- **Storage:** Encrypt keys and restrict access.
- **Rotation:** Replace keys according to a defined policy.
- **Revocation:** Disable compromised or expired keys.
- **Destruction:** Securely remove keys when no longer required.

5. Example Python Code

The following example uses the cryptography library to protect data using authenticated encryption:

```
from cryptography.fernet import Fernet



key = Fernet.generate_key()

cipher = Fernet(key)



encrypted = cipher.encrypt(b"Confidential data")

decrypted = cipher.decrypt(encrypted)



print(decrypted.decode())
```

Output:

```
Confidential data
```

This is a demonstration only. A production system must store and protect the key separately; generating a new key each run would make previously encrypted data unrecoverable unless the old key was retained securely.

6. Security Analysis

Poor key management can compromise an entire cryptographic system, even when strong algorithms are used. If an attacker obtains the encryption key, they may be able to decrypt protected information.

Proper access control, key rotation, auditing, secure backups, and separation of duties reduce these risks.

Conclusion

Secure key management protects cryptographic keys throughout their lifecycle. Encryption, restricted access, regular rotation, and secure storage help prevent unauthorized access and data breaches.

---

5. Optimization of Secure File Transfer Protocol (SFTP-like System)
A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.
Fix errors in packet handling and integrity verification.
Optimize throughput for large file transfers (≥100 MB).
Compare the optimized system with insecure file transfer methods and justify the improvements.

ANSWER:

1. Introduction

Secure file transfer systems protect files while transmitting them between computers. SFTP is a file transfer protocol that operates over SSH and provides encrypted communication.

2. Identify the Errors

Common problems include:

- Incorrect encryption and decryption synchronization.
- Incomplete or duplicated packets.
- Incorrect handling of partial file reads and writes.
- Missing integrity verification.
- Inefficient transfer chunk sizes.
- Repeated connection establishment.

3. Fix the Errors

- Use established protocols and cryptographic libraries.
- Handle partial reads and writes correctly.
- Verify the size and integrity of transferred files.
- Detect incomplete transfers and retry safely.
- Use authenticated encryption or a secure protocol such as SFTP.

4. Optimize Transfer Speed

- Transfer data in chunks rather than loading entire files into memory.
- Reuse secure connections.
- Use appropriate buffer sizes.
- Avoid unnecessary encryption and decryption operations.
- Measure throughput and memory usage.

Example Python code for chunked integrity checking:

```
import hashlib



def file_hash(path):

    h = hashlib.sha256()

    with open(path, "rb") as f:

        for chunk in iter(lambda: f.read(1024 * 1024), b""):

            h.update(chunk)

    return h.hexdigest()



print(file_hash("largefile.bin"))
```

This calculates a SHA-256 hash without loading the entire file into memory. In a real transfer, compare the sender's and receiver's hashes using a trusted channel or authenticated protocol.

5. Comparison with Insecure File Transfer

Feature	Secure Transfer	Insecure Transfer
Confidentiality	Encryption protects data	Data may be exposed
Integrity	Integrity checks detect modification	Modification may go undetected
Authentication	Supports secure authentication	May lack reliable authentication
Performance	Has cryptographic overhead	May have less overhead

Feature	Secure Transfer	Insecure Transfer
Security	Protects sensitive information	Vulnerable to interception

6. Security Analysis

Secure transfer protocols provide confidentiality and integrity while protecting against unauthorized access. Chunked processing improves memory efficiency, especially for files larger than 100 MB.

Performance should be measured under realistic network conditions. Disabling encryption to increase speed is not a safe optimization.

Conclusion

A secure file transfer system can achieve better throughput and reliability through chunked processing, connection reuse, and efficient implementations while retaining encryption, authentication, and integrity verification.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation